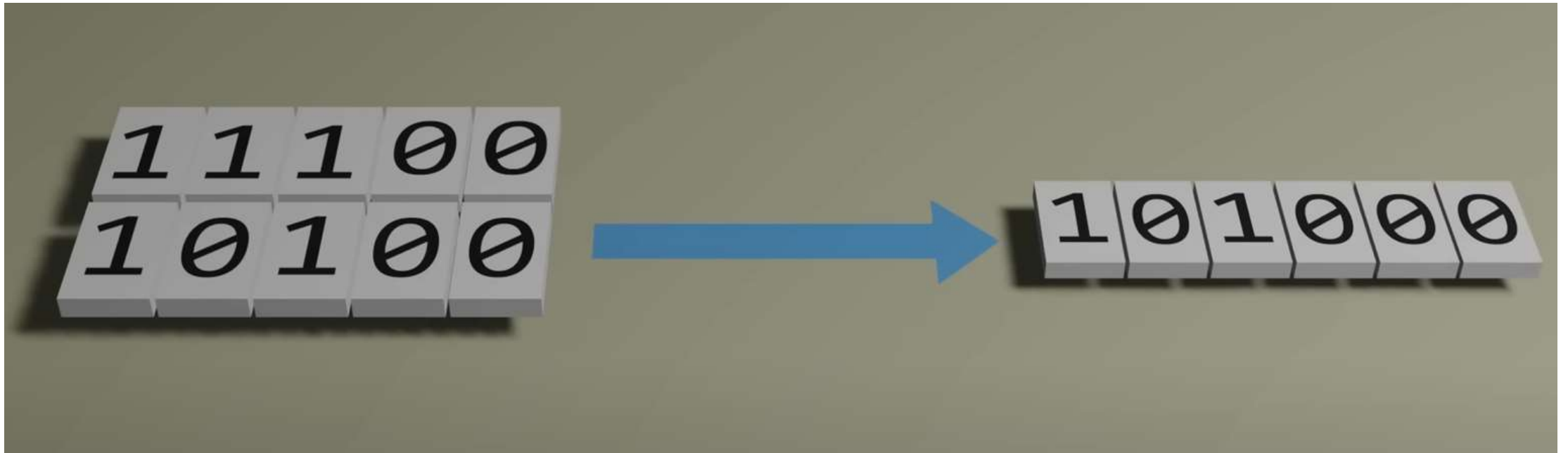
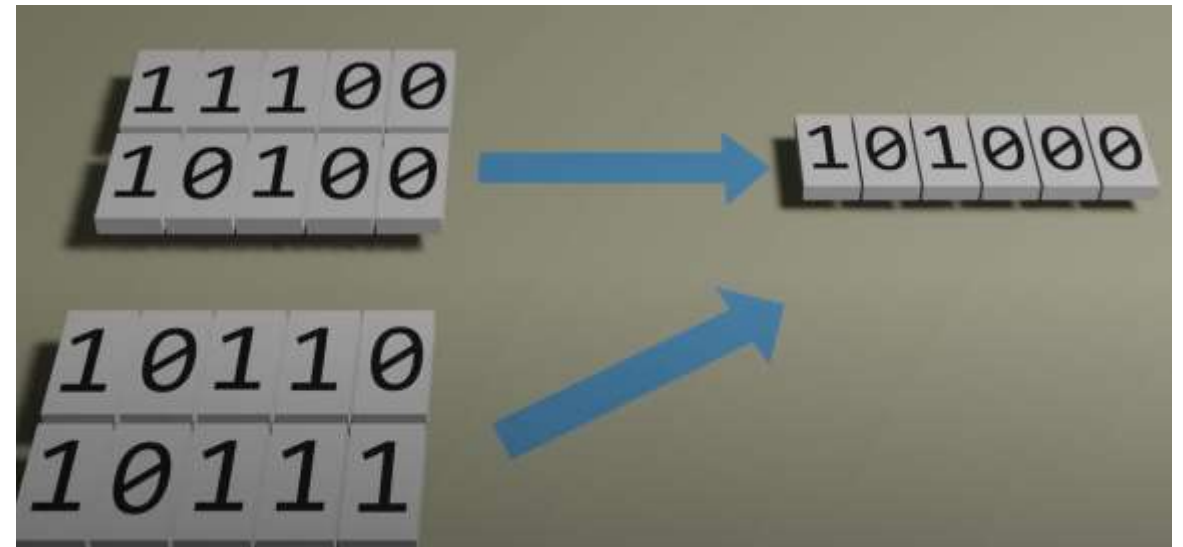
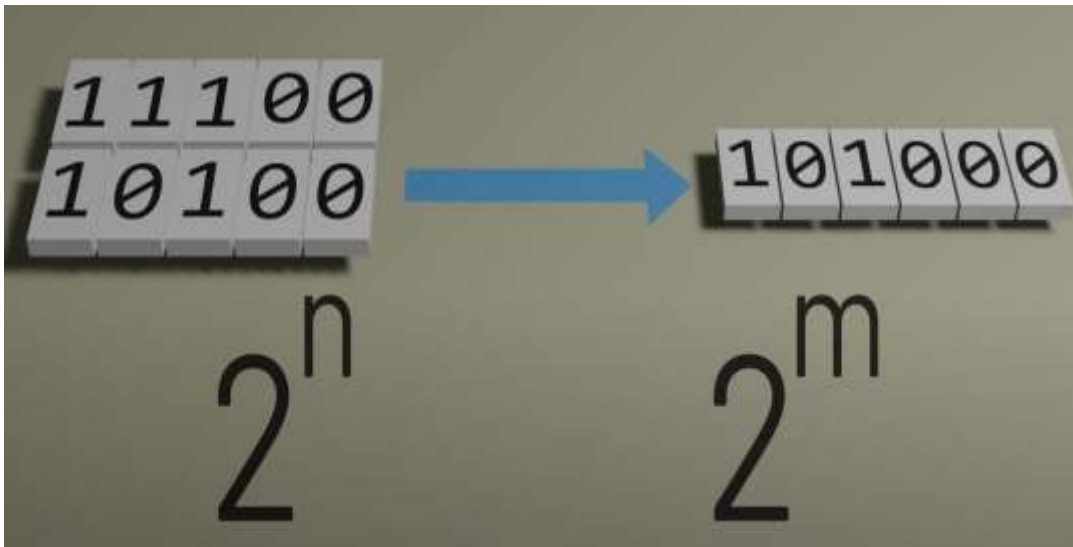


# Compression



# Compression

- If  $n > m$ , then according to **pigeonhole principle**, there must be at least one encoded string which is matched with more than one original string.



# Compression

- For a lossless compression, multiple string matching with single encoded string is not possible.
- There must be at least one input which will be matched with the output having size bigger than input
- Huffman encoding is a lossless text file compression technique

# Huffman encoding

Given a long string of characters, find a way to encode these characters into binary codes such that the length of the binary representation of the string is minimized

- It assigns variable-length **prefix free** codes to input characters
- Code lengths depend on the frequencies of corresponding characters
- Target is to find a prefix free code so that the encoded string has the smallest length

# Huffman encoding (example)

- Encoding 1: a = 0 b = 1 c = 01
- Decode the message based on the previous encoding: 0101 ?

# Huffman encoding (example)

- Encoding 1: a = 0 b = 1 c = 01
- Decode the message based on the previous encoding: 0101 ?

|         |        |        |       |
|---------|--------|--------|-------|
| 0 1 0 1 | 0 1 01 | 01 0 1 | 01 01 |
| a b a b | a b c  | c a b  | c c   |

- This encoding is not prefix free (code for **a** is found in the prefix of code **c**)

# Huffman encoding (example)

Text to encode: **abcc**

- **Some prefix free coding options are:**
  - Encoding1: a = 0, b = 10, c = 11
    - Encoded string: 0101111 (**length: 7**)
  - Encoding2: c = 0, a = 10, b = 11
    - Encoded string: 101100 (**length: 6**)
  - Encoding3:.....
  - Encoding4:.....
- Huffman encoding finds the encoding where the encoded string has the smallest length

# Huffman encoding

- **To achieve prefix free code**, the binary tree representing the encoding will have symbols at the leaves only, the intermediate nodes will not represent any symbols
- **To achieve the shortest encoded string**, the highest frequency symbols will be closer to the root and lower frequency symbols will be in the deeper levels of the tree



# Huffman encoding (Greedy solution)

1. Create a leaf node for each unique character and build a min heap of all leaf nodes (Min Heap is used as a priority queue. The value of frequency field is used to compare two nodes in min heap. Initially, the least frequent character is at root)
2. Extract two nodes with the minimum frequency from the min heap.
3. Create a new internal node with a frequency equal to the sum of the two nodes frequencies. Make the first extracted node as its left child and the other extracted node as its right child. Add this node to the min heap.
4. Repeat steps#2 and #3 until the heap contains only one node. The remaining node is the root node and the tree is complete.

Let us understand the algorithm with an example:

**Source:** <https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/>

# Optimality of Huffman encoding

- **Proof by induction:**
- Base case:  $n \leq 2$ , our algorithm and Huffman tree are same (Only one tree is possible with 1 or 2 nodes)
- We will assume the claim to be true for any sequence of  $n-1$  frequencies and prove that it holds for any  $n$  frequencies

# Optimality of Huffman encoding

- Assume, Huffman tree is  $H$  and our algorithm generates  $G$
- Our goal is to prove  $\text{cost}(H) = \text{cost}(G)$  [Not:  $H = G$ ]
- Let  $f_1 \leq f_2 \leq f_3 \dots \leq f_n$  are the frequencies for  $n$  symbols
- There's an optimal tree where the two smallest frequency symbols mark siblings (Proof given later), so:  $f_1$  and  $f_2$  are neighbors in  $H$
- Our algorithm also forms a tree  $G$ , where  $f_1$  and  $f_2$  are neighbors

# Optimality of Huffman encoding

- Now remove  $f_1$  and  $f_2$  and make their parent a new leaf with frequency  $(f_1+f_2)$ . Using this idea, we obtain  $H'$  and  $G'$  from  $H$  and  $G$ .
- $H'$  and  $G'$  has  $n-1$  elements and according to our hypothesis for induction:
  - $\text{cost}(H') = \text{cost}(G') \dots (1)$
- Now:
  - $\text{cost}(G') = \text{cost}(G) - (f_1+f_2) \dots (2)$
  - $\text{cost}(H') = \text{cost}(H) - (f_1+f_2) \dots (3)$
- Using (1), (2) and (3):
  - $\text{cost}(H) = \text{cost}(H') + f_1 + f_2 = \text{cost}(G') + f_1 + f_2 = \text{cost}(G)$

# Proof of the Claim

- **Claim: There's an optimal tree where the two smallest frequency symbols mark siblings**
- Let,  $w, y$  are the lowest frequency symbols. (They have the biggest depth in the tree)
- If  $w, y$  are not the siblings, 3 cases can happen:
- $w$  has a neighbour  $z$ :
  - Swap  $z$  with  $w$ :  $\text{cost}(T') \leq \text{cost}(T)$
- $w$  is the only child:
  - Replace  $w$  with its parent:  $\text{cost}(T') < \text{cost}(T)$  [Contradiction, as Huffman tree is optimal]
- $w$  has a subtree with  $z$  as leaf ( $z$  is in deeper level than  $w$ ):
  - Swap  $w$  with  $z$ :  $\text{cost}(T') < \text{cost}(T)$  [Contradiction, as Huffman tree is optimal]

# Visualization Tool

- <https://cmps-people.ok.ubc.ca/ylucet/DS/Huffman.html>